COSC204 Computer Systems

Lecture 16: Device drivers

Zhiyi Huang

School of Computing, University of Otago

Overview

Last lecture: file systems

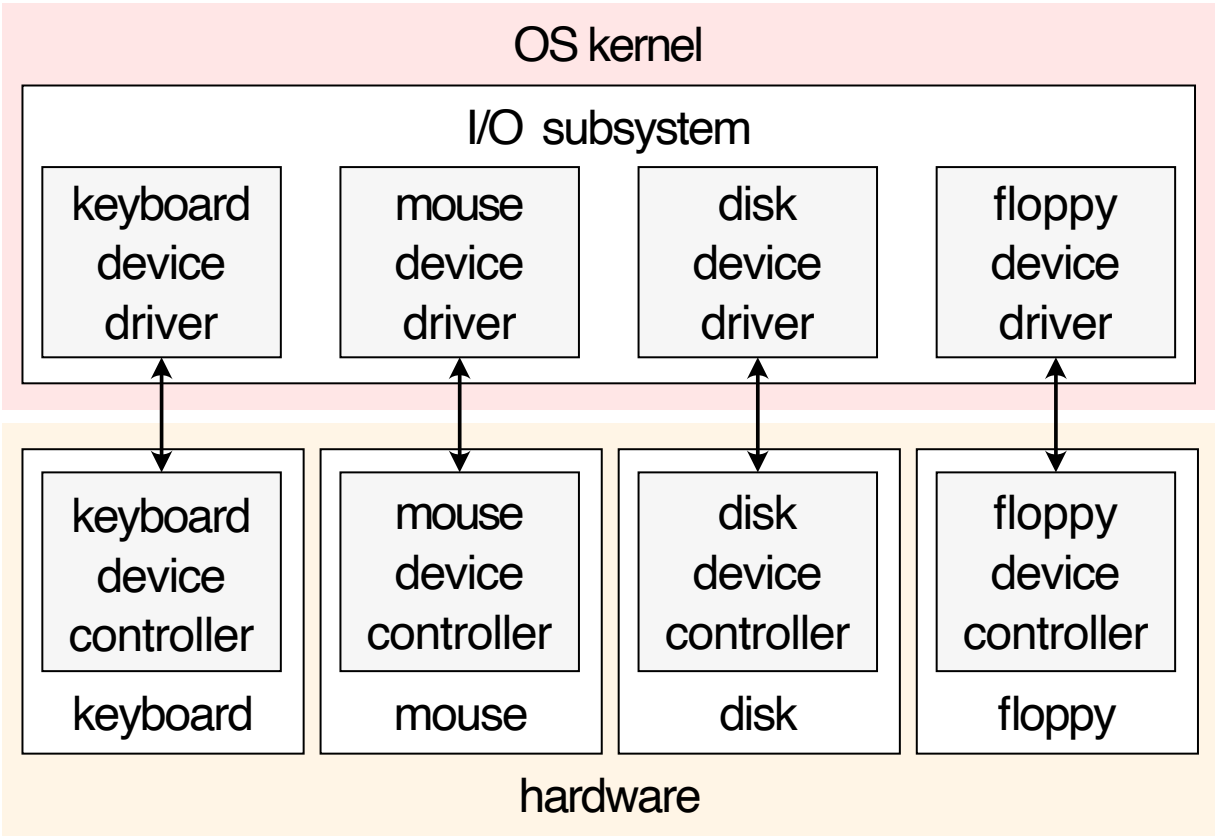
- The concept of a file
- File access patterns
- Directory structure
- File system implementation
- Disk allocation methods

This lecture: device drivers

- Device controllers and drivers
- Types of I/O devices
- I/O models
- I/O scheduling and buffering
- User interface for I/O devices

Device drivers are components of OS kernels

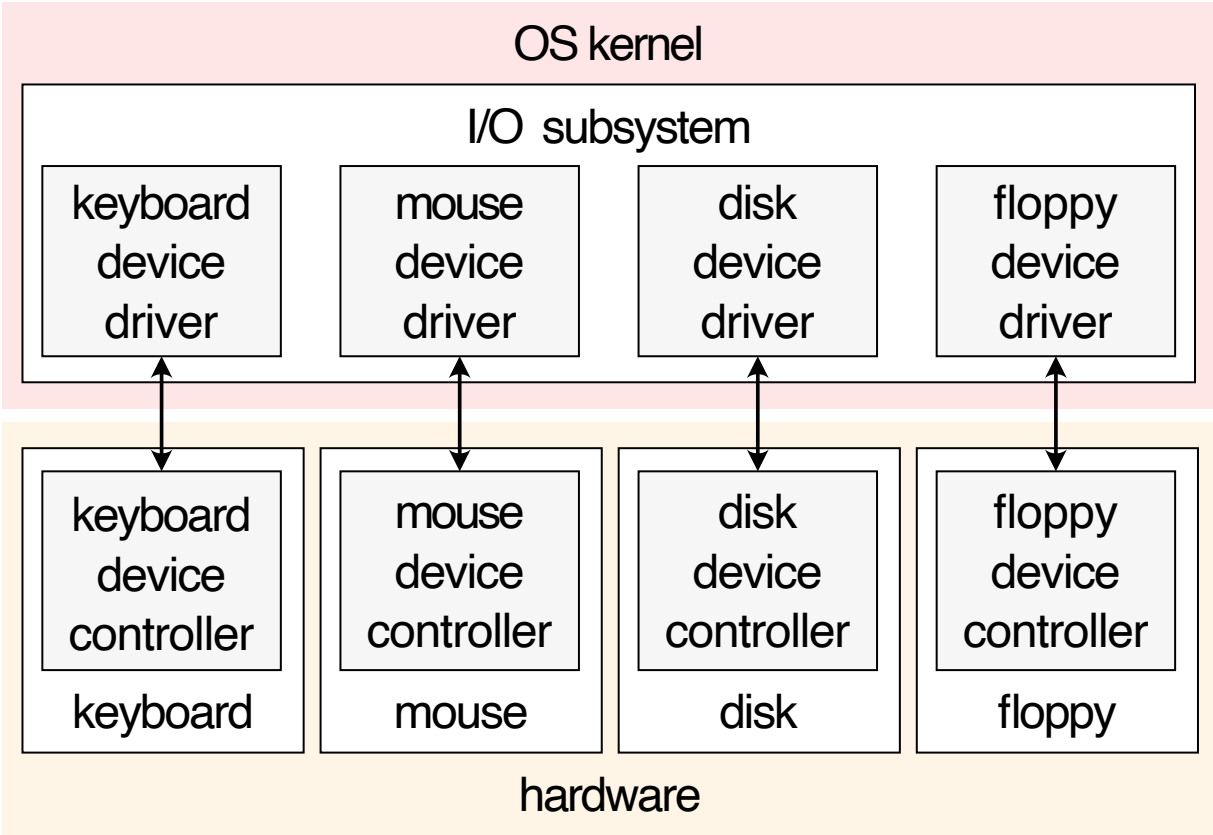
A device driver is software running in kernel mode that controls the operations of a device with the device-specific information (i.e., knowledge of the device controller).



Device controllers run within hardware devices

A device controller is a hardware unit on a device that can control and monitor a device. The CPU communicates with the controller via the driver of the device.

There is a one-to-one mapping between device drivers and device controllers.



An example disk controller on an SSD device

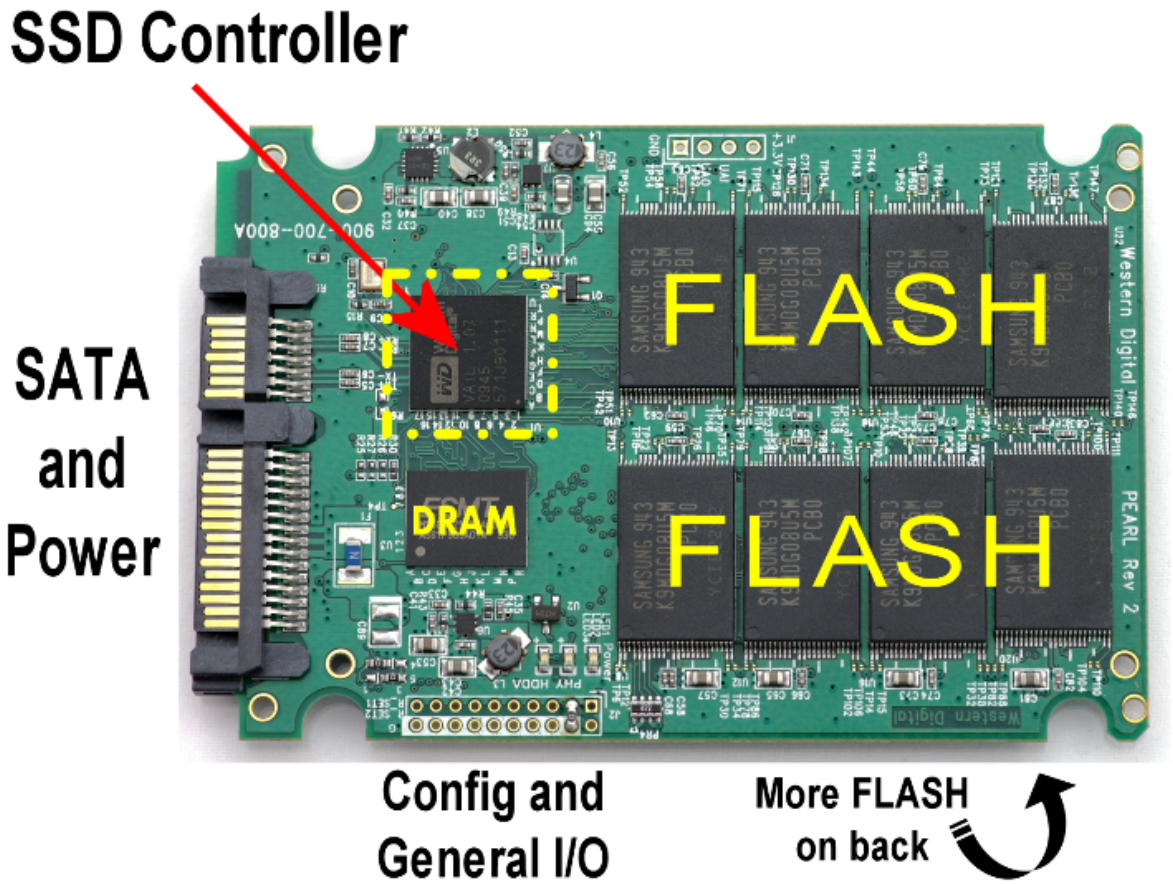


Image source: `https://medium.com/@songchai.d01/how-to-perform-ssd-forensics-part-i-1158dd3975e8`

Types of I/O device

I/O devices vary widely, along several dimensions.
Here are some key distinctions:

- Character stream devices versus block devices.
- Sequential versus random access devices.
- Synchronous versus asynchronous devices.
- Shareable or dedicated devices.

Unix-flavoured OSs such as Linux provide a consistent interface for devices under `/dev`. For example, our ramdisk device is named `/dev/memdrv`. Each device is treated as a special file under `/dev` that can be handled with file operations such as `read()` or `write()`.

Block devices

The standard commands sent by the device driver to a block device are:

- *Read* block
- *Write* block
- *Seek* block (for random-access block devices).

Block devices need to optimize the sequence of the read/write requests in order to improve performance, e.g., I/O scheduling.

Character devices

The standard commands sent by the device driver to a character device:

- *Get* character.
- *Put* character.

Character devices can support various preprocessing facilities:

- Buffering: e.g., accepting keyboard input a line at a time.
- Editing of buffers: e.g., special treatment of backspace character.

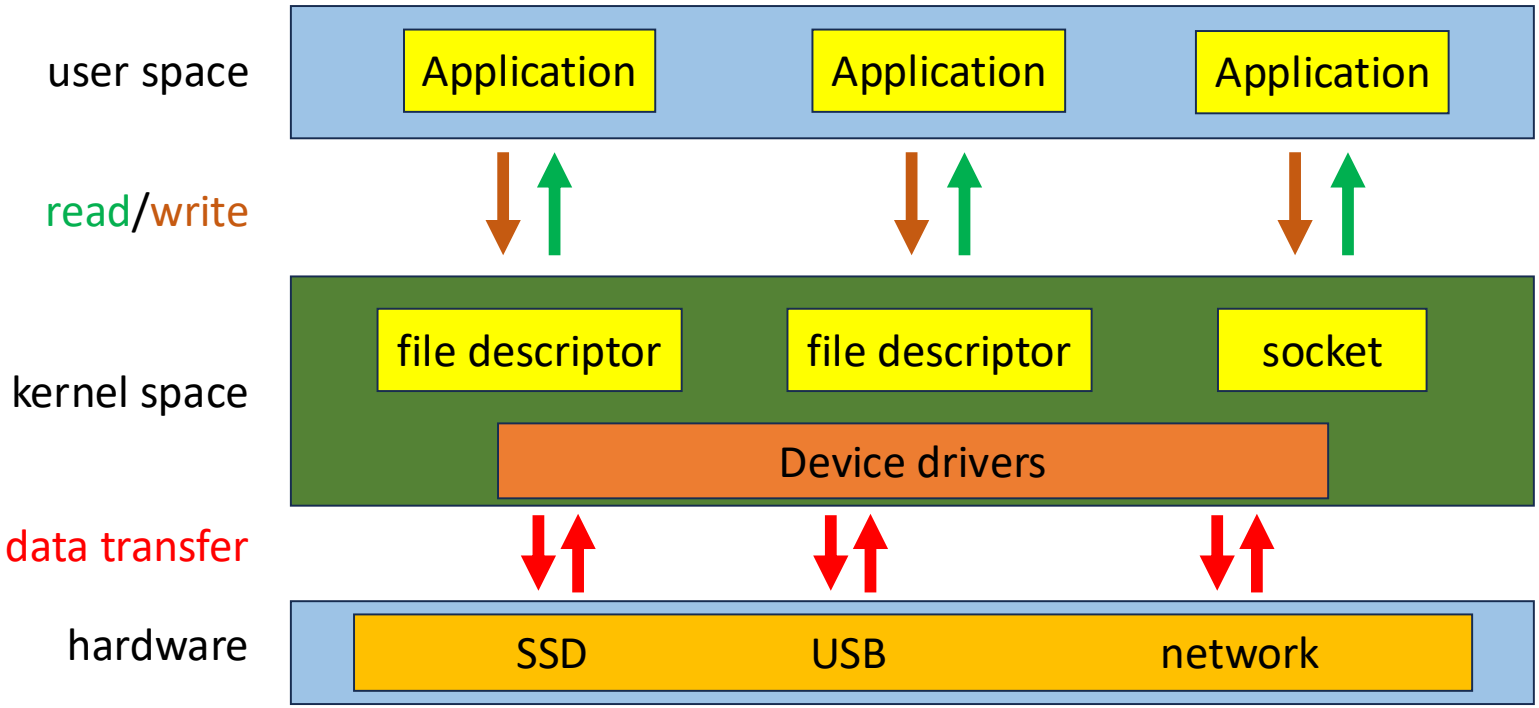
I/O models for system calls

I/O system calls can use different I/O models.

- Blocking I/O: If a process issues an I/O system call, it waits until the I/O is completed.
- Non-blocking I/O: If a process issues an I/O system call, it waits for a fixed (but small) interval, and returns (possibly with some data still untransferred).
- Asynchronous I/O: If a process issues an I/O call, it does not wait for the I/O to be completed, but the OS informs the process with a signal when the I/O is completed.

Difference between nonblocking and asynchronous I/O?
In nonblocking I/O, the process has to keep trying the system call until the I/O is completed.
In asynchronous I/O, the process only issues the call once, and OS will ensure the I/O is completed and inform the process.

Depicting various different I/O models



I/O scheduling helps optimise access to devices

At any one time, there might be several requests for I/O waiting to be serviced.

- We could just do these in first-come-first-served order.
- Disadvantages with this scheme?
  Let's say the disk arm is currently near the beginning of a disk, and there are 5 I/O requests: 3 for places near the beginning and 2 for places near the end...

I/O buffering can improve performance

I/O buffers are used in main memory to improve the performance of file systems.

When a data block is used often, instead of fetching it repeatedly from the secondary storage, it is more efficient to keep it in memory and write to the secondary storage only once, after all of the modifications to that block are complete.

A simple RAMDISK driver

There is an intuitive mapping between file-related system calls and the driver functions.

```
1 struct file_operations temp_fops = {
2     .owner =      THIS_MODULE,
3     .llseek =     temp_llseek,
4     .read =       temp_read,
5     .write =      temp_write,
6     .unlocked_ioctl = temp_ioctl,
7     .open =       temp_open,
8     .release =    temp_release,
9 };
```

What happens after `open()` is called?

```
1 int temp_open(struct inode *inode, struct file *filp)
2 {
3   // check if process has the permission to access;
4   // suspend calling process if the device is busy
5   // or used by another process;
6   // otherwise record the ID of the calling process
7   // so that the device is owned by the process;
8   // initialise or reset the device.
9 }
```

For our RAMDISK device, we haven't done anything like that yet.

What happens after `read()` is called?

```
1  ssize_t temp_read(struct file *filp,
2              char __user *buf,
3              size_t count, loff_t *f_pos){
4    // find the location of data;
5    // ask the controller to copy data to kernel kbuf
6    // copy data to user space kbuf -> buf
7    copy_to_user (buf, kbuf, count);
8    *f_pos += count; // update pointer
9    return count; // return the number of bytes read
10 }
```

What happens after `write()` is called?

```
1  ssize_t temp_write(struct file *filp,
2          char __user *buf,
3          size_t count, loff_t *f_pos){
4    // copy data from user space buf -> kbuf
5    copy_from_user (kbuf, buf, count);
6    // find the location of data;
7    // ask the controller to copy data from kernel kbuf
8    *f_pos += count; // update pointer
9    return count; // return the number of bytes written
10 }
```

What happens after `close()` is called?

```
1 int temp_release(struct inode *inode,
2                   struct file *filp){
3   // release the device: remove the ID of the process;
4   // check if there is a waiting process to wake up;
5   // undo anything done at open(), e.g. free memory.
6 }
```

Readings

For material relevant to this lecture, you should read Chapter 13 (Sections 3–4).